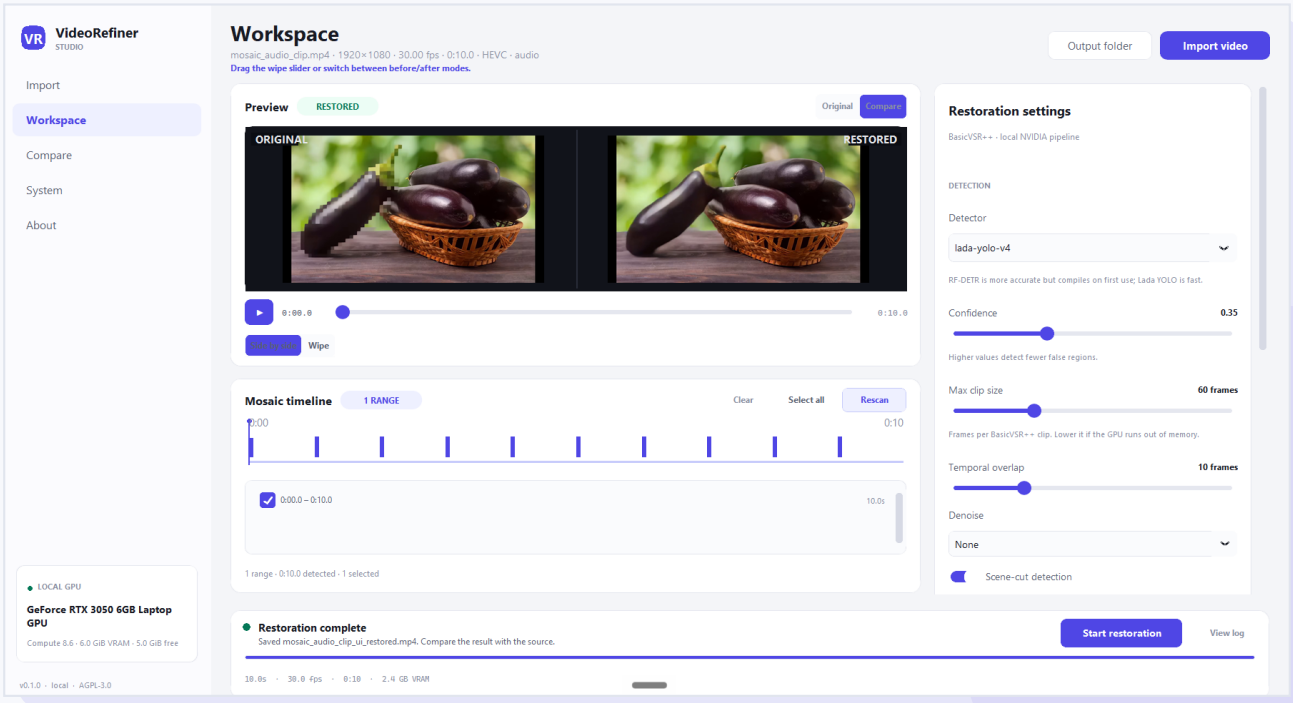
VR

VideoRefiner Studio

A local Windows app for mosaic detection and video restoration.

Find mosaic areas, restore them with AI, compare the result, and export safely.
Everything stays on your computer. No cloud. No account. No telemetry.

PRODUCT GUIDE | TECHNICAL HANDOFF | TEST REPORT



WORKSPACE PREVIEW / SOURCE + RESTORED COMPARISON

START HERE

Contents

How to use this PDF Click a heading or page number to jump. Use the PDF bookmarks panel for a section list. The setup checklist on page 8 can be filled in.

01 Project at a Glance . . . 3

02 Feature Map . . . 4

03 Architecture at a Glance . . . 5

04 Processing Pipeline . . . 6

05 UI Tour . . . 7

06 Setup and Launch . . . 8

07 Quick-Start Workflow . . . 9

08 Models, Codecs, and Export Rules . . . 10

09 Test Results and Evidence . . . 11

10 Performance and VRAM Guidance . . . 12

11 Troubleshooting . . . 13

12 Limitations and Next Steps . . . 14

13 Licensing and Attribution . . . 15

14 Appendix . . . 16

What is interactive?
The contents list, footer links, and repository links are clickable. The PDF has bookmarks for every section and topic. The setup checklist contains fillable checkboxes.

The short version
VideoRefiner Studio is a working local prototype. It scans a video, restores selected ranges with BasicVSR++, and exports a new file without touching the source. All three workflow tests pass.

01 Project at a Glance

VideoRefiner Studio is a small desktop app for a focused job: find mosaic areas, restore them, and check the result before saving. It hides the heavy AI work behind a clear, light interface.

100%	3/3	AGPL
Local processing	Workflow tests pass	Open-source license

The problem

Mosaic blur hides parts of a video. Repairing it by hand means finding each area, fixing it frame by frame, and checking the result. That takes time and is easy to get wrong.

The solution

The app samples the video, marks likely mosaic ranges on a timeline, and restores only the ranges you choose. You compare source and result at the same moment, then export a new file. The source file is never overwritten.

Who it is for

- People who edit their own local video and want a simple desktop tool.
- Reviewers who need a fast before/after check without a cloud upload.
- Developers who want a readable example of a Jasna desktop front end.

Design principles

- **Private by default.** Video, frames, and settings stay on this computer.
- **Safe by default.** Every export gets a new path and the source is read-only.
- **Clear by default.** One main workflow, visible progress, and plain-language errors.
- **Fast where it matters.** GPU detection, TensorRT, and smart range rendering.

02 Feature Map

Six features define the current build. Each one supports the same simple loop: import, scan, select, restore, compare, export.

01
Local scan
Samples one frame per second. The detector scores each sample and turns the scores into one-second ranges.

02
Range or whole video
Restore only the selected ranges, or process the full video with a codec of your choice.

03
BasicVSR++ restore
Uses the Lada BasicVSR++ model on the NVIDIA GPU to rebuild detail inside the selected areas.

04
Before / after compare
Switch between Original, Compare, Side by side, and Wipe. Compare the same timestamp.

05
Safe export
Writes a new file with a unique name. Preserves audio when the source has it. Never overwrites the source.

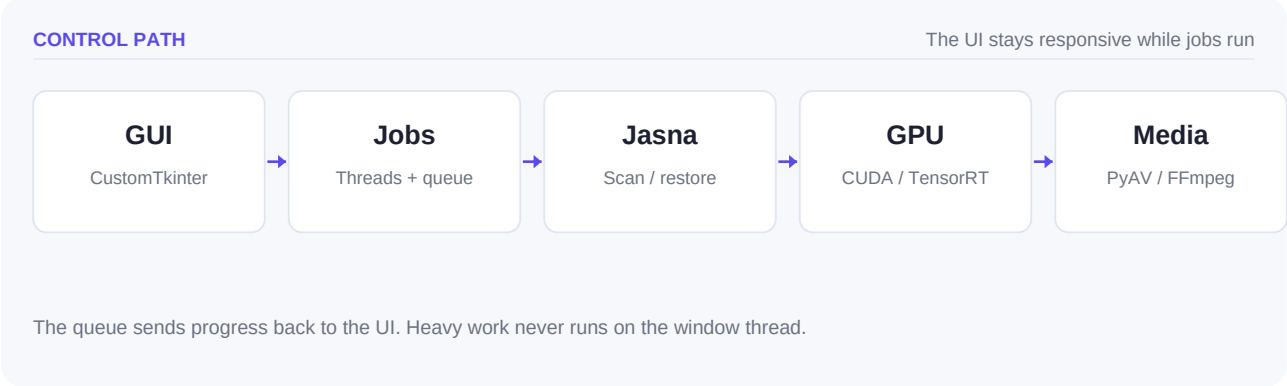
06
System check
Shows GPU, VRAM, CUDA, TensorRT, FFmpeg, model, and VLC status in one place.

What is not in this build

Kept out on purpose
No cloud processing, no account, no telemetry, and no paid API. LTX, UNet 4x, and SD 1.5 models are not included. This build uses BasicVSR++ only. Audio is muxed into exports but is not previewed.

03 Architecture at a Glance

The app separates the window from the work. The window stays responsive while background threads scan and restore video.



The parts

Part	What it does	Why it matters
Desktop UI	CustomTkinter views, settings, timeline, player, and status footer.	One clear place to control the job.
Job layer	Scan and restore threads send events through a queue.	The window does not freeze during long work.
Jasna engine	Pinned upstream engine for detection and restoration.	The AI logic stays upstream and unmodified.
GPU runtime	CUDA, PyTorch, TensorRT, and the compiled detector.	Fast local inference on an NVIDIA card.
Media layer	PyAV and FFmpeg for frames, muxing, and preview.	Readable video and preserved audio.

Why this design works

- The UI thread only draws and reacts. It does not decode or run AI.
- Workers own the long tasks and report progress as messages.
- Settings are small and local, so a restart is cheap and predictable.
- The source file is treated as read-only input, not as a working file.

04 Processing Pipeline

The workflow is a straight line. Each step gives you something visible before the next step starts.

1	**Import**	FFprobe reads the file. The header shows name, size, frame rate, duration, codec, and audio status. The source is only read.
2	**Scan**	The detector looks at one frame per second. Each sample gets a score. Scores become short ranges on the timeline.
3	**Select**	Choose ranges on the timeline or press Select all. You can change confidence and clip settings before restoring.
4	**Restore**	BasicVSR++ processes the selected clips on the GPU. Overlap and crossfade help hide the joins between clips.
5	**Compare and export**	Watch the result at the same timestamp. Save a new file with a unique name, then open it in your player.

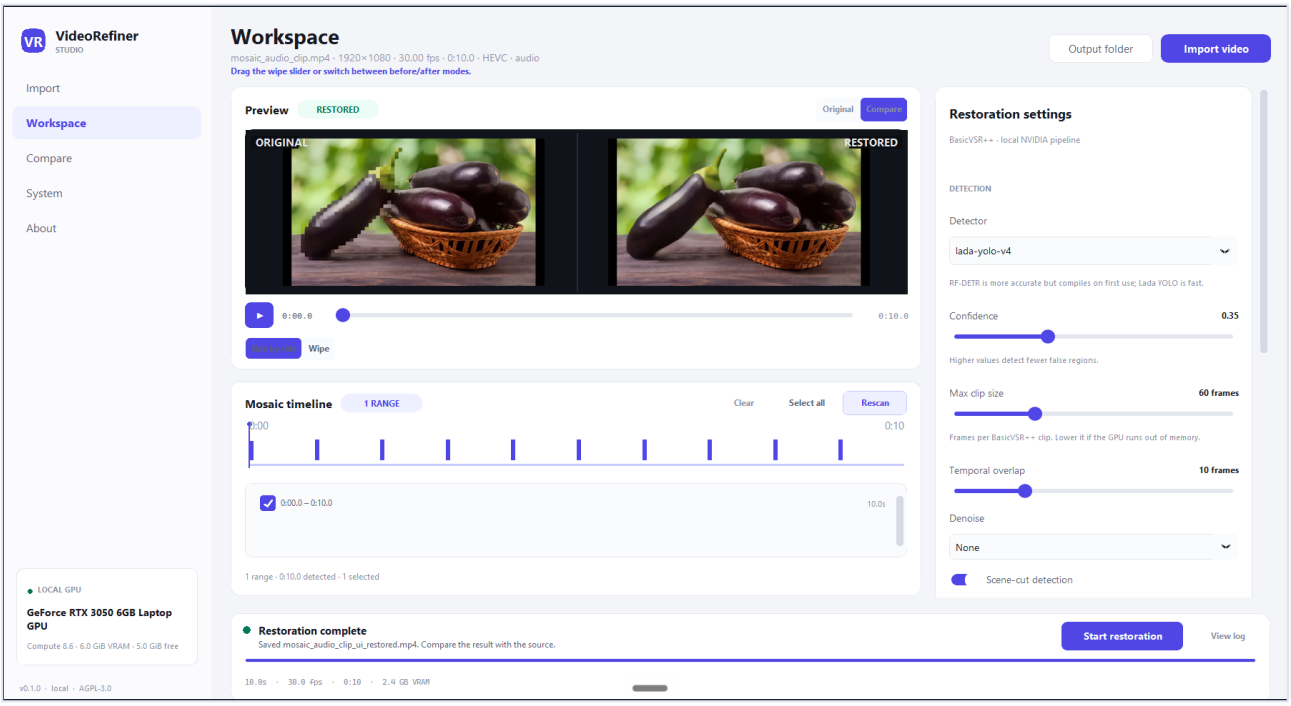
Smart range rendering

For selected ranges, the app restores the marked parts and streams the rest of the video from the source. This keeps untouched scenes from being re-encoded. It also keeps the source codec, which is why selected-range mode supports only H.264, HEVC, and AV1 sources.

> **The important rule**
> Selected ranges keep the source codec. Whole video uses the codec you pick. If the source codec cannot be smart-rendered, the app disables selected-range restore and asks you to use whole video.

05 UI Tour

The main workspace is built around one screen. The source and result are always visible together, and the start button stays in the footer.



Workspace after a completed selected-range restore. Screenshot generated by scripts/ui_screenshot.py.

What to notice

Area	What it shows	What you can do
Left sidebar	Import, Workspace, Compare, System, About, and the local GPU card.	Move between views and check free VRAM.
Header	File name, resolution, frame rate, duration, codec, and audio.	Confirm you loaded the right file.
Preview	Source and restored frames, player, and compare modes.	Switch views, play, scrub, or wipe.
Timeline	Detected ranges and selected ranges.	Select all, clear, rescan, or pick a subset.
Settings	Detector, confidence, clip size, overlap, denoise, and scene detection.	Tune quality and GPU load.
Footer	Status, progress, stats, and Start restoration.	Start, cancel, or open the log.

06 Setup and Launch

The current workspace is already set up. This page is also useful for a clean machine or a new clone.

Already set up?

Double-click **run.bat**, or run this command from the project folder:

Run the app

```
.venv\Scripts\python.exe main.py
```

Fresh setup

Install **uv**, **Git for Windows**, and **FFmpeg**, then run:

Bootstrap command

```
powershell -ExecutionPolicy Bypass -File scripts\bootstrap.ps1
```

Need	Version or note
Windows	Windows 10 or 11, 64-bit.
GPU	NVIDIA GPU with a current driver. This build was tested on an RTX 3050 6 GB Laptop GPU.
Python	3.13, created by uv inside the project .venv.
Media tools	FFmpeg and ffprobe on PATH. Version 9.0 was used for this build.
Disk	Room for the virtual environment and model files. Models are several hundred MB.

Setup checklist

These checkboxes are fillable in a PDF reader that supports forms, such as Adobe Acrobat or Microsoft Edge.

- [ ] Windows PC with an NVIDIA GPU
- [ ] Current NVIDIA driver installed
- [ ] uv installed and available on PATH
- [ ] Git for Windows installed
- [ ] FFmpeg and ffprobe on PATH
- [ ] scripts\bootstrap.ps1 finished successfully
- [ ] run.bat opens the app and the GPU card turns green

Settings live in **%LOCALAPPDATA%\VideoRefinerStudio**. Set **VIDEOREFINER_CONFIG_DIR** to use a different folder. Useful links: uv, Git, FFmpeg, Jasna.

07 Quick-Start Workflow

This is the normal path for a new video. The first scan can be slower because the detector may compile a TensorRT engine.

1	**Import**	Use Import video or drop a file on the import screen.
2	**Choose scope**	Keep Selected ranges for a fast repair, or choose Whole video.
3	**Scan**	Press Scan. Watch the timeline fill with ranges.
4	**Select**	Press Select all or tick the ranges you want. Check the count and duration.
5	**Tune**	Lower confidence if mosaics are missed. Raise it if normal areas are flagged.
6	**Start**	Press Start restoration in the footer. Keep the window open while it works.
7	**Check and save**	Compare at the same timestamp. Use Output folder if needed, then open the new file.

Quick tuning guide

Symptom	Try this
Mosaic areas are missed	Lower Confidence in small steps, then rescan.
Normal areas are flagged	Raise Confidence, then rescan.
GPU runs out of memory	Lower Max clip size, close other GPU apps, or use H.264 whole video.
Selected ranges are disabled	Use an H.264, HEVC, or AV1 source, select a range, or switch to Whole video.

Good first test
Use a short 10-second clip. Confirm the preview, compare modes, audio, and output file before starting a long video.

08 Models, Codecs, and Export Rules

Three local model files do the AI work. The app chooses safe defaults for a 6 GB GPU.

Model	Role	License	Default?
lada_mosaic_restoration_model_generic_v1.2.pth	BasicVSR++ restoration	AGPL-3.0	Yes
lada_mosaic_detection_model_v4_fast.pt	Lada YOLO v4 fast detector	AGPL-3.0	Yes
rfdetr-v6-large.onnx	Optional RF-DETR large detector	Apache-2.0	No

What the models do

The **detector** finds likely mosaic areas. The **restoration model** rebuilds detail inside those areas. The detector is the fast part. The restoration model does the heavy visual work.

Codec rules

Mode	Codec behavior	When to use it
Selected ranges	Always keeps the source codec. Supports H.264, HEVC, and AV1 sources.	Fast repair of specific ranges.
Whole video	Uses the codec you choose: H.264, HEVC, or AV1.	Unsupported source codec or a full re-encode.

Export rules

- The app writes a new file with a unique name. An existing file or the source is never overwritten.
- The default suffix is _**restored**. Change it in settings if you want a different tag.
- Source audio is muxed into the export when present.
- The comparison player does not play audio. This is expected.

> **Quality setting**
> CQ controls quality. A lower CQ means higher quality and a larger file. The allowed range depends on the codec.

09 Test Results and Evidence

The test opens the real desktop window, uses an isolated settings folder, and runs the same code paths as a person using the app.

Case	Output	Audio	Check	Evidence
Selected ranges + compare	10.0 s HEVC	No source audio	PASS	Preview difference 2.32
Whole video	4.9 s H.264	Preserved	PASS	Audio present in output
Selected ranges + audio	10.0 s HEVC	Preserved	PASS	Audio + difference 2.32

What the test proves

- The app loads a video and shows the correct file facts in the header.
- Scanning finds ranges and enabling the Start button works.
- Selected-range restore writes a complete output file.
- Whole-video restore preserves audio.
- The restored preview is visibly different from the source (mean difference 2.32).
- Compare mode and playback update the live UI.

Reproduce it

Test command

```
.venv\Scripts\python.exe scripts\ui_workflow_test.py
```

The test prints PASS at the end. Output files are written under **test_output\ui_test** with unique names.

Small note

The test terminal may show one harmless Tkinter finalizer warning when the test window is torn down. The workflow still finishes with PASS, and the warning does not appear during normal app use.

10 Performance and VRAM Guidance

The GPU does the heavy work. The biggest limits are video size, clip size, and how much VRAM is free.

Factor	Effect	What to do
Resolution	4K needs much more VRAM than 1080p.	Start at 1080p. Use shorter test clips.
Video length	Long videos take longer to scan and restore.	Test the workflow on 10 seconds first.
Detector	RF-DETR is more accurate but slower and larger.	Keep Lada YOLO for speed on 6 GB.
Max clip size	Large clips need more VRAM at once.	Lower it if the GPU runs out of memory.
First run	TensorRT may compile the detector once.	Wait for the first scan to finish.

6 GB VRAM starting point

- Use 1080p video and the default Lada YOLO detector.
- Keep Max clip size at 60 frames. Try 30 or 24 if memory is tight.
- Close games, browsers with many tabs, and other GPU apps before a long job.
- If selected-range mode is not needed, try whole-video H.264 for a smaller memory footprint.
- Scene-cut detection is on by default. It can add work on videos with many cuts.

> **Speed is local**
> There is no upload or queue wait. Speed depends on this computer's GPU, driver, video, and settings.

What good looks like

The 10-second HEVC test clip restored in about 20-30 seconds on the test machine after warm-up. Treat this as a rough guide, not a promise. Longer and larger videos take longer.

11 Troubleshooting

Start with the System view and the log. Most problems have a clear cause and a simple fix.

Problem	What to try
App does not start	Open %LOCALAPPDATA%\VideoRefinerStudio\app.log and read the last error.
App says CUDA is not available	Update the NVIDIA driver and confirm the .venv has the CUDA build of PyTorch.
Start restoration is disabled	Select a range, use H.264/HEVC/AV1 for selected mode, or switch to Whole video.
GPU runs out of memory	Lower Max clip size, close other GPU apps, or use H.264 whole-video export.
Scan is very slow	The first run may compile TensorRT. Later runs reuse the engine.
Preview is black or frozen	Move the playhead, switch Original/Compare, or reload the video.
No preview audio	Expected. Audio is preserved in the export but not played in the compare view.
Wrong or stale settings	Delete settings.json or set VIDEOREFINER_CONFIG_DIR to a fresh folder.

Where things are

Item	Path
Log	%LOCALAPPDATA%\VideoRefinerStudio\app.log
Settings	%LOCALAPPDATA%\VideoRefinerStudio\settings.json
Models	model_weights\ inside the project
Test outputs	test_output\ui_test\ inside the project

System view
The System view checks the GPU, VRAM, CUDA, TensorRT, FFmpeg, model files, and optional VLC. Open it before changing code or reinstalling packages.

12 Limitations and Next Steps

This is a working local build, not a finished commercial product. The limits are known and the next steps are clear.

Current limits

- **Windows and NVIDIA only.** There is no CPU-only or AMD path in this build.
- **6 GB VRAM is a real limit.** 4K and high-frame-rate video can run out of memory.
- **The first run can be slow.** TensorRT may compile a detector engine once.
- **No installer.** The app runs from this folder with the project virtual environment.
- **No audio preview.** Audio is preserved in exports only.
- **BasicVSR++ only.** LTX, UNet 4x, and SD 1.5 are not part of this build.

Possible next steps

Next step	Why it helps
Signed Windows installer	Makes setup and first launch much easier for non-developers.
Batch queue	Lets a user prepare several videos and walk away.
Presets	Saves good settings for 1080p, 4K, animation, or low-VRAM runs.
Better progress estimates	Shows time remaining from real job history.
More compare tools	Add zoom, frame step, and saved snapshots.
More detectors	Offer a quality/speed choice without editing files.

Project status
Version 0.1.0 is a working local prototype with a complete import-to-export loop and passing workflow tests.

13 Licensing and Attribution

VideoRefiner Studio is free and open source under AGPL-3.0-only because it uses Jasna. Keep the notices with the app.

The main license

The project is distributed under the **GNU Affero General Public License version 3.0 only**. The full text is in **LICENSE**. If you share the app, provide the corresponding source and keep the attribution.

Component	License	Note
VideoRefiner Studio	AGPL-3.0-only	Desktop front end and UI.
Jasna 0.12.0	AGPL-3.0-only	Pinned upstream engine in vendor/jasna.
Lada models	AGPL-3.0	BasicVSR++ restoration and YOLO detector.
RF-DETR large	Apache-2.0	Optional detector model.
PyTorch / TensorRT	BSD / NVIDIA	Runtime and inference tools.
FFmpeg / PyAV	LGPL/GPL / BSD	Media read, write, and mux.

What to keep

- **LICENSE**, **NOTICE**, and **THIRD_PARTY_NOTICES.md**.
- Jasna's notices under **vendor/jasna**.
- Model provenance and license notes for any model you redistribute.
- A clear note that the app is local and has no telemetry.

> **Simple rule**
> Edit only media you have permission to edit. If you share the app or a modified version, share the source and keep the license notices.

Project links: Jasna / AGPL-3.0

14 Appendix

A quick map of the files, commands, and words used in this guide.

File map

Path	Role
main.py	DPI setup, logging, environment defaults, app entry point.
videorefiner/app.py	Main window, views, timeline, compare player, status footer.
videorefiner/engine.py	Scan and restore workers, GPU checks, codec rules.
videorefiner/media.py	Video probe, frame preview, compare composition, playback.
videorefiner/settings.py	JSON settings and config-directory override.
videorefiner/widgets.py	Scroll area, cards, timeline, pills, and UI parts.
scripts/bootstrap.ps1	Clean-clone setup for Windows.
scripts/download_models.py	SHA-256 checked model downloads.
scripts/ui_workflow_test.py	Three-case end-to-end UI test.
vendor/jasna	Pinned upstream Jasna source, not modified.

Commands

Task	Command
Run	.venv\Scripts\python.exe main.py
Run tests	.venv\Scripts\python.exe scripts\ui_workflow_test.py
Screenshot	.venv\Scripts\python.exe scripts\ui_screenshot.py
Models	.venv\Scripts\python.exe scripts\download_models.py

Words used in this guide

Word	Simple meaning
Mosaic	A blocky blur that hides part of a video.
Codec	The format used to compress video, such as H.264 or HEVC.
CQ	A quality setting. Lower means better quality and a bigger file.
TensorRT	An NVIDIA tool that makes AI models run faster.
VRAM	Memory on the graphics card.
Smart render	Re-encode only the changed ranges and copy the rest.